

Backend — Documentation

Choix techniques

Le backend d'ETNAir est un serveur **Node.js 24** utilisant le framework **Express 5**. Le projet est intégralement écrit en modules ECMAScript natifs (`"type": "module"` dans le `package.json`), ce qui apporte une syntaxe d'import moderne mais demande quelques précautions, notamment pour les tests.

L'accès à la base de données passe par **Prisma 6**, un ORM moderne qui combine une excellente expérience développeur (auto-complétion, types générés) avec des performances solides. Pour PostgreSQL, le projet utilise le nouvel adapter natif `@prisma/adapters-pg` qui remplace l'ancien moteur Rust et offre une meilleure intégration. Les mots de passe sont hashés avec **bcrypt**, et les tokens d'authentification sont des **JWT** signés avec une clé secrète stockée en variable d'environnement.

Pour la validation des requêtes entrantes, le projet utilise **express-validator** qui permet de déclarer les règles de validation directement dans la définition des routes. Les uploads d'images sont gérés par **Multer** avec un stockage sur disque. La documentation auto-générée est exposée via **Swagger UI**, alimentée par les commentaires JSDoc dans les fichiers de routes. Enfin, les tests sont écrits avec **Jest** et `supertest` pour la simulation des requêtes HTTP.

Organisation du code

Le code source est organisé sous `api/src/` selon une architecture classique en couches. Le point d'entrée est `server.js` à la racine, qui configure Express, monte les routes et démarre le serveur sur le port 3000.

Le sous-dossier `controllers/` contient la logique métier de chaque domaine fonctionnel. On y trouve un contrôleur par grande entité : `authController` pour l'inscription et la connexion, `UserController` pour les opérations sur les utilisateurs, `listingController` pour les annonces, `bookingController` pour les réservations, et `favoriteController` pour les favoris.

Le sous-dossier `routes/` déclare les routes Express et les associe aux fonctions des contrôleurs. Chaque route applique en amont les middlewares appropriés : la validation des inputs avec `express-validator`, l'authentification avec `authMiddleware`, et l'upload de fichiers avec Multer pour les endpoints qui en ont besoin.

Le sous-dossier `middleware/` regroupe les middlewares transversaux. Le fichier `auth.js` contient le middleware qui decode le token JWT depuis le header `Authorization`, vérifie sa validité, charge l'utilisateur correspondant en base, et l'attache à la requête. Le fichier `upload.js` configure Multer avec un stockage sur disque pointant vers `/app/uploads`, et applique des filtres sur le type MIME (jpeg, png, gif, webp) ainsi qu'une limite de taille à 5 Mo.

Les utilitaires sont dans `utils/`. Le fichier `hash.js` enveloppe `bcrypt` avec deux fonctions `hashPassword` et `comparePassword`. Le fichier `jwt.js` expose `generateToken(userId)` et `verifyToken(token)` qui utilisent la `JWT_SECRET` lue dans l'environnement.

Le script de seed se trouve dans `scripts/seed.js` et génère les données fictives au premier démarrage. Enfin, les tests sont dans `tests/`, et les mocks Jest dans `__mocks__`.

Endpoints de l'API

L'API expose plusieurs domaines fonctionnels, tous accessibles sous le préfixe `/api` après proxification par le frontend.

Authentification

Deux endpoints permettent à un utilisateur de créer un compte ou de se connecter. `POST /auth/register` accepte un objet avec un email, un mot de passe (minimum 6 caractères), un prénom et un nom, ainsi qu'un rôle optionnel (`tenant` ou `owner`). La validation est stricte : email au bon format, mot de passe assez long, prénom et nom non vides. Si le compte est créé avec succès, un token JWT est immédiatement renvoyé pour que l'utilisateur soit connecté automatiquement. `POST /auth/login` prend simplement un email et un mot de passe, retourne un token JWT en cas de succès, et un 401 sinon.

Utilisateurs

L'endpoint `GET /utilisateurs` retourne la liste complète des utilisateurs et ne demande pas d'authentification (utile pour des écrans d'administration). `GET /utilisateurs/:id` exige un token et renvoie le détail d'un utilisateur avec ses annonces associées (incluant leurs images). Cet endpoint est utilisé notamment par le dashboard pour afficher les annonces publiées par l'utilisateur courant. `PUT /utilisateurs/:id` et `DELETE /utilisateurs/:id` permettent respectivement de modifier et supprimer un compte.

Annonces

L'endpoint principal `GET /annonces` retourne la liste paginée des annonces. Il accepte des paramètres query comme `page`, `limit`, `city` et `maxPrice` pour filtrer les résultats. Chaque annonce retournée contient son propriétaire (avec ses informations de contact), ses images, et un flag `isBooked` qui indique si une réservation confirmée et active existe actuellement. Ce flag est calculé via un `_count` Prisma qui compte les bookings respectant les conditions `status: 'confirmed'` et `endDate: { gte: now }`.

`GET /annonces/:id` retourne le détail complet d'une annonce avec ses images, son propriétaire et ses avis. `POST /annonces` est protégé et permet à un propriétaire de créer une nouvelle annonce. La requête est de type `multipart/form-data` pour pouvoir inclure jusqu'à cinq images. `PUT /annonces/:id` permet la modification, gère aussi bien l'ajout de nouvelles images que la suppression d'images existantes (via un paramètre `removeImageIds`). Enfin `DELETE /annonces/:id` supprime l'annonce, ce qui déclenche en cascade la suppression de ses images, bookings et avis associés.

Favoris

Le système de favoris expose quatre endpoints. `GET /favoris` renvoie la liste complète des annonces favorites de l'utilisateur courant avec toutes leurs informations. `GET /favoris/ids` est plus léger : il renvoie uniquement les IDs, ce qui est utilisé par le frontend pour savoir quelles cartes afficher avec un cœur rouge sans avoir à charger toutes les données. `POST /favoris/:listingId` ajoute une annonce, et `DELETE /favoris/:listingId` la retire. Tous ces endpoints requièrent une authentification.

Réservations

`POST /reservations` permet à un locataire de créer une demande de réservation. Le contrôleur vérifie plusieurs choses avant d'enregistrer : que les dates demandées ne sont pas dans le passé, qu'il y a au moins une nuit, que l'utilisateur n'est pas le propriétaire de sa propre annonce, et qu'aucune autre réservation en attente ou confirmée ne chevauche les dates demandées. Si tout est bon, la réservation est créée avec le statut `pending` et un `cancelDeadline` calculé à 48h avant la date d'arrivée.

`GET /reservations/mes-reservations` retourne les réservations de l'utilisateur courant en tant que locataire, et `GET /reservations/recues` retourne les réservations reçues sur les annonces de l'utilisateur en tant que propriétaire. `PUT /reservations/:id/confirm` permet à un propriétaire d'accepter une demande en attente (et ne peut être appelée que par le propriétaire de l'annonce, sur une résa au statut `pending`). `PUT /reservations/:id/annuler` peut être appelée soit par le locataire soit par le propriétaire, à condition que la réservation ne soit pas déjà annulée et que le délai d'annulation ne soit pas dépassé. Enfin `GET`

`/reservations/disponibilite/:listingId` retourne les périodes où l'annonce est déjà occupée, ce qui permet au frontend de griser les dates dans le calendrier de réservation.

Endpoints utilitaires

`GET /health` est un simple endpoint qui retourne `{ "status": "OK" }` et sert de healthcheck pour Kubernetes (livenessProbe et readinessProbe). `GET /api-docs` expose la documentation Swagger générée à partir des commentaires JSDoc. `GET /uploads/<file>` sert les images uploadées de manière statique.

Authentification et sécurité

Le flux d'authentification est le suivant. Lors d'un login réussi, le serveur génère un JWT avec le `userId` comme payload et le retourne au frontend, qui le stocke dans le `localStorage`. À chaque requête ultérieure, l'intercepteur Axios ajoute automatiquement ce token dans le header `Authorization: Bearer <token>`. Le `authMiddleware` du backend extrait le token, le décode avec `verifyToken`, charge l'utilisateur correspondant en base via `prisma.user.findUnique`, et attache son ID à `req.userId` pour que les contrôleurs puissent l'utiliser.

Les mots de passe sont hashés avec `bcrypt` (10 rounds) avant insertion en base. Les comparaisons utilisent `bcrypt.compare` qui est sécurisé contre les timing attacks. La `JWT_SECRET` est injectée en variable d'environnement et ne doit jamais être committée dans le repo. Bien que le token n'ait pas d'expiration native côté serveur, le frontend impose une expiration de 1 heure côté client, ce qui force l'utilisateur à se reconnecter régulièrement.

Gestion des uploads

Le middleware `upload.js` configure Multer avec un stockage sur disque (`multer.diskStorage`). Le dossier de destination est `/app/uploads` dans le conteneur, monté en volume Docker vers `./api/uploads` sur l'hôte. Cela garantit que les images persistent entre les redémarrages et même entre les rebuilds de l'image Docker. Les noms de fichiers sont préfixés par un timestamp et un suffixe aléatoire pour éviter les collisions.

Un filtre MIME accepte uniquement les formats jpeg, jpg, png, gif et webp, ce qui évite qu'un utilisateur malveillant n'uploadé un exécutable. La taille de chaque fichier est limitée à 5 Mo, et un maximum de 5 fichiers peut être envoyé dans une seule requête. Les images sont ensuite servies de manière statique via `app.use('/uploads', express.static(...))`.

Tests Jest

L'approche choisie pour les tests est de **tout mocker au niveau Prisma**, ce qui évite d'avoir besoin d'une vraie base de données pour exécuter les tests. Le fichier `src/__mocks__/prisma.js` exporte un objet `prisma` factice où chaque méthode (`findUnique`, `create`, `update`, etc.) est une `jest.fn()` qui peut être configurée test par test.

Le mock est appliqué automatiquement grâce à `moduleNameMapper` dans `jest.config.js`. Toute importation de `lib/prisma.js` (que ce soit depuis un contrôleur, un middleware ou un test) est redirigée vers le mock. De la même manière, le SDK AWS S3 (utilisé par une route legacy qui n'est plus active) est mocké pour éviter l'erreur "module not found" en CI.

Comme le projet utilise ESM nativement, Jest doit être lancé avec `node --experimental-vm-modules`, et les imports `jest`, `describe`, `it`, etc. ne sont pas globaux : ils doivent être importés explicitement depuis `@jest/globals`.

La suite de tests actuelle couvre quatre domaines. Le fichier `auth.test.js` vérifie que l'inscription accepte un email valide mais rejette les emails malformés ou les mots de passe trop courts, et que le login retourne un token avec les bons identifiants. Le fichier `utilisateurs.test.js` teste les endpoints CRUD avec authentification. Le fichier `annonces.test.js` couvre la pagination, le filtre par ville, la création et la suppression. Enfin `booking.test.js` valide les flux de réservation, de confirmation, d'annulation et la disponibilité, et vérifie que le flag `isBooked` reflète correctement l'état de la base.

Variables d'environnement

Le serveur attend plusieurs variables au démarrage. `DATABASE_URL` est l'URL de connexion à PostgreSQL (par exemple `postgres://etnair_user:etnair_pass@db:5432/etnair_db` dans le contexte Docker). `JWT_SECRET` est la clé qui signe les tokens, elle doit être longue et imprévisible en production. `PORT` indique le port HTTP, par défaut 3000. `NODE_ENV` peut valoir `production` ou `test`, ce qui influence le chargement de dotenv et certains logs.

Cycle de vie au démarrage

Le script `docker-entrypoint.sh` est exécuté à chaque démarrage du conteneur API. Il enchaîne trois étapes. D'abord, `npx prisma migrate deploy` applique toutes les migrations en attente sur la base, ce qui garantit que le schéma est à jour. Ensuite, `node src/scripts/seed.js` peuple la base avec les données initiales si elle est vide (l'existence de l'utilisateur sentinelle `seed@etnair.com` sert de marqueur pour éviter de re-seeder). Enfin, `node server.js` démarre effectivement l'API.

Cette séquence est idempotente : on peut redémarrer le conteneur autant de fois qu'on veut, les migrations ne seront appliquées qu'une fois et le seed ne s'exécutera que si nécessaire.